

Дообучение: математика и методы

vlm-recipes · подзадача 2

1. Что именно оптимизируется

Языковая модель задаёт распределение над последовательностями токенов через авторегрессионную факторизацию:

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t}).$$

Обучение — минимизация отрицательного логарифма правдоподобия, то есть кросс-энтропии между предсказанным распределением и наблюдаемым токеном:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}).$$

Здесь и возникает первая практическая развилка. Сумма идёт по всем позициям, включая промпт пользователя, системную инструкцию и результаты вызовов инструментов. Модель, обученная так, учится в том числе *порождать* вопросы пользователя и *выдумывать* вывод инструментов.

Маскирование ограничивает сумму множеством A позиций, принадлежащих репликам ассистента:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \frac{1}{|A|} \sum_{t \in A} \log p_{\theta}(x_t | x_{<t}).$$

Существенно, что $x_{<t}$ по-прежнему включает *весь* контекст. Маскирование убирает позиции из функции потерь, но не из обусловливания: модель видит промпт, просто не получает градиента за его предсказание. Технически это реализуется значением -100 в метках, которое `cross_entropy` пропускает.

Отсюда же диагностический признак: если доля обучаемых токенов $|A|/T$ мала (единицы процентов), почти весь вычислительный бюджет батча уходит на прямой проход по тексту, за который нет градиента.

2. Полное дообучение и его цена

Обновляются все P параметров. Помимо самих весов в памяти живут градиенты и состояния оптимизатора. Для AdamW со смешанной точностью:

веса bf16	$2P$
мастер-копия fp32	$4P$
градиенты	$2P$
моменты AdamW m, v (fp32)	$8P$
итого	$\approx 16P$ байт

Для $P = 9 \cdot 10^9$ это около 144 ГБ — без учёта активаций. Отсюда и растут все остальные методы: они атакуют разные слагаемые этой суммы.

3. LoRA: низкоранговая адаптация

3.1. Конструкция

Идея опирается на наблюдение Aghajanyan и др. о низкой внутренней размерности задач дообучения: приращение весов ΔW , нужное для адаптации к узкой задаче, хорошо приближается матрицей малого ранга.

Для слоя с весами $W_0 \in \mathbb{R}^{d \times k}$ приращение параметризуется произведением

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k),$$

а прямой проход становится

$$h = W_0 x + \gamma B A x, \quad \gamma = \frac{\alpha}{r}.$$

Число обучаемых параметров падает с dk до $r(d+k)$. При $d = k = 4096$ и $r = 32$ это $2,6 \cdot 10^5$ вместо $1,7 \cdot 10^7$ — в шестьдесят четыре раза меньше.

3.2. Инициализация

A инициализируется случайно, B — нулём. Тогда в начале обучения $\Delta W = 0$, и модель стартует ровно из претрейна: адаптер ничего не портит до первого шага.

Возникает вопрос, почему не наоборот. Выпишем градиенты, обозначив $g = \partial L / \partial h$:

$$\frac{\partial L}{\partial B} = \gamma g (Ax)^\top, \quad \frac{\partial L}{\partial A} = \gamma B^\top g x^\top.$$

При A случайном и $B = 0$ градиент по A обнуляется множителем $B^\top$, но градиент по B содержит $Ax \neq 0$ — значит B сдвигается на первом же шаге, после чего оживает и A . При $A = 0$ и B случайном картина зеркальна: сдвигается A , следом оживает B .

Обе схемы обучаются. Сломан только третий случай — обе матрицы нулевые: тогда оба градиента нулевые навсегда. Ноль нужен ровно в одной из матриц, чтобы дать $\Delta W = 0$ на старте, и случайность в другой, чтобы разорвать симметрию.

Выбор между двумя рабочими схемами определяется не запуском обучения, а его динамикой. Науои и др. (2024) показывают, что схемы не эквивалентны: при случайном A и нулевом B допустима бо́льшая скорость обучения без потери устойчивости. Интуиция в том, что на первом шаге A работает как фиксированная случайная проекция входа, а B учится уже в спроецированном пространстве; размерности матриц различны, поэтому различны и масштабы обновлений.

3.3. Почему память экономится в разы

Замороженные W_0 не требуют ни градиентов, ни моментов. Состояния оптимизатора нужны только для BA , то есть для доли процента параметров. Веса остаются в памяти (их нужно читать на прямом проходе), но три остальных слагаемых из таблицы выше исчезают почти целиком.

4. Масштаб: почему α/r плох при больших рангах

Это место стоит разобрать, потому что оно объясняет распространённое наблюдение «ранг выше 32 ничего не даёт».

Пусть после нескольких шагов элементы A и B имеют порядок σ_A и σ_B и приблизительно независимы. Элемент произведения есть сумма r слагаемых:

$$(BA)_{ij} = \sum_{s=1}^r B_{is} A_{sj}, \quad \text{Var}[(BA)_{ij}] \approx r \sigma_B^2 \sigma_A^2.$$

Среднеквадратичная величина элемента растёт как $\sqrt{r}$. С множителем $\gamma = \alpha/r$ вклад адаптера в выход имеет масштаб

$$\gamma \cdot \sqrt{r} \sigma_B \sigma_A = \frac{\alpha \sigma_B \sigma_A}{\sqrt{r}} \xrightarrow{r \rightarrow \infty} 0.$$

При $\gamma = \alpha/r$ вклад адаптера *затухает* как $1/\sqrt{r}$. Увеличивая ранг, вы одновременно глушите адаптер, и два эффекта гасят друг друга. Правильный масштаб —

$$\gamma = \frac{\alpha}{\sqrt{r}},$$

при нём величина вклада от r не зависит. Это и есть **rsLoRA** (rank-stabilized LoRA, Kalajdzievski, 2023); в PEFT включается флагом `use_rslora=True`.

Практический вывод: при $\gamma = \alpha/r$ разумный диапазон рангов кончается около 32. С rsLoRA ранги 64 и 128 начинают приносить реальный прирост. Стоит это ничего, поэтому включать следует всегда.

5. DoRA: разделение величины и направления

Эмпирическое наблюдение Liu и др. (2024): при полном дообучении норма столбца весов и его направление меняются *независимо* — бывают большие повороты при почти неизменной норме и наоборот. LoRA такой свободы не даёт: одно приращение BA управляет обоими сразу.

DoRA разлагает веса явно:

$$W = m \odot \frac{V}{\|V\|_c},$$

где $\|\cdot\|_c$ — норма по столбцам, $m \in \mathbb{R}^{1 \times k}$ — обучаемый вектор величин, V — направление. Дообучение применяет низкоранговое приращение только к направлению:

$$W' = m \odot \frac{W_0 + BA}{\|W_0 + BA\|_c}.$$

Обучаются m, A, B . Нормировка развязывает две степени свободы, и на низких рангах ($r \leq 16$) DoRA устойчиво обходит LoRA. Цена — вычисление нормы на каждом шаге, около 20% скорости.

6. Квантование базы: NF4 и QLoRA

6.1. Почему не обычный int4

Равномерная сетка из 16 уровней плоха для весов нейросети, потому что они распределены приблизительно нормально: большинство значений сосредоточено у нуля, а равномерные уровни тратят разрешение на хвосты.

NF4 (NormalFloat4) строит уровни как квантили стандартного нормального распределения. Уровни q_i выбираются так, чтобы каждый интервал между ними нёс равную вероятностную массу:

$$q_i \propto Q_{\mathcal{N}}\left(\frac{i + 0.5}{2^b}\right), \quad i = 0, \dots, 2^b - 1,$$

где $Q_{\mathcal{N}}$ — квантильная функция. Для входа, распределённого нормально, такая сетка информационно-теоретически оптимальна.

6.2. Блоки и двойное квантование

Веса нормируются поблочно: блок из 64 значений делится на свой масштаб $c = \max |w|$, и уже отнормированное квантуется в NF4. Масштабы хранятся отдельно и сами занимают память: при блоке 64 и fp32-масштабе это $32/64 = 0,5$ бита на параметр.

Двойное квантование сжимает и масштабы: они квантуются в 8 бит блоками по 256, что снижает накладные расходы до $\approx 0,127$ бита на параметр. Итого QLoRA хранит вес примерно в 4,127 бита.

6.3. Что происходит на прямом проходе

$$h = \text{dequant}_{\text{bf16}}(W^{\text{NF4}})x + \gamma B A x.$$

Умножение выполняется в bf16: веса разжимаются поблочно на лету.

Отсюда важное следствие: **QLoRA экономит память, но не ускоряет**. Разжатие добавляет работу, поэтому шаг обучения медленнее, чем у LoRA в bf16. Брать QLoRA следует тогда и только тогда, когда bf16-база не помещается.

Обучаемые A , B остаются в bf16 — квантованию подвергается только замороженная база, иначе градиенты потеряли бы разрешение.

7. Ортогональные приёмы

7.1. NEFTune

К эмбедингам входа при обучении добавляется равномерный шум:

$$x'_{\text{emb}} = x_{\text{emb}} + \varepsilon, \quad \varepsilon \sim \mathcal{U}\left(-\frac{\alpha}{\sqrt{Ld}}, \frac{\alpha}{\sqrt{Ld}}\right),$$

где L — длина последовательности, d — размерность эмбединга. Нормировка на $\sqrt{Ld}$ делает величину возмущения независимой от длины примера.

Эффект — регуляризация: модель перестаёт цепляться за точную форму обучающих ответов. На малых выборках прирост заметный, стоимость нулевая. Обычное значение $\alpha = 5$.

7.2. Слитые ядра (Liger)

Наивное вычисление лосса материализует логиты размера $B \times L \times |V|$. При словаре $|V| \sim 150,000$ это доминирующее слагаемое в памяти активаций. Слитые ядра считают проекцию в словарь и кросс-энтропию чанками, не храня полную матрицу логитов. Экономия тем больше, чем крупнее словарь; на качество не влияет никак.

7.3. GaLore: низкий ранг в градиенте, а не в весах

Альтернатива LoRA, сохраняющая полноранговое обучение весов. Градиент $G_t \in \mathbb{R}^{m \times n}$ проецируется в низкоранговое подпространство, и состояния оптимизатора живут уже там:

$$\tilde{G}_t = P^\top G_t, \quad P \in \mathbb{R}^{m \times r},$$

где P — первые r левых сингулярных векторов G_t , пересчитываемые раз в несколько сотен шагов. Обновление возвращается в исходное пространство: $\Delta W = -\eta P \rho(\tilde{G}_t)$.

Смысл в том, что ограничение накладывается на оптимизатор, а не на выразительность модели: веса меняются полноранговым образом. Память моментов падает в m/r раз.

8. Что выбрать

Метод	Когда	Цена
full	максимум качества, ресурсы есть	$16P$ байт памяти
lora	базовый выбор	r выше 32 бесполезен
rsloa	всегда вместо lora	нет
dora	низкие ранги, тонкая настройка стиля	—20% скорости
qlora	база не помещается в bf16	медленнее lora
qdora	то же плюс низкий ранг	медленнее всех
galore	нужно полноранговое обучение при ограниченной памяти	SVD раз в N шагов

9. Передний край

Направления, которые стоит держать в виду, но не брать по умолчанию.

LoRA+ (Hayou и др., 2024). Показано, что оптимальные скорости обучения для A и B различаются на порядок из-за разной размерности: $\eta_B = \lambda \eta_A$ при $\lambda \sim 16$. Одинаковая скорость, принятая в стандартной LoRA, замедляет сходимость.

PiSSA (Meng и др., 2024). Вместо случайной инициализации A, B берутся главные компоненты W_0 : $W_0 \approx U_r S_r V_r^\top$, $A = \sqrt{S_r} V_r^\top$, $B = U_r \sqrt{S_r}$, а остаток замораживается. Адаптер сразу начинает с наиболее значимого подпространства и сходится быстрее.

VeRA (Koriczko и др., 2024). A и B общие для всех слоёв и заморожены; обучаются только два вектора масштаба на слой. Даёт сокращение числа параметров ещё на порядок при сопоставимом качестве.

Spectrum и **LISA**. Оба выбирают подмножество слоёв для полного обучения: Spectrum — по отношению сигнал/шум в спектре весов, LISA — случайно с перевыбором каждые K шагов. Промежуточный вариант между LoRA и полным дообучением.

10. Практика замера

Сравнивать методы имеет смысл по четырём числам одновременно: доля обучаемых параметров, значение функции потерь, время шага и пиковая память. Метод, выигравший по потерям ценой пятикратного времени, на итерациях проиграет.

Обязателен замер до обучения: без базовой линии непонятно, что дал метод, а что модель умела и так. Реализовано в `vlmkit.compare.sweep` — каждый прогон начинается с чистой загрузки, иначе адаптер предыдущего метода протекает в следующий замер.